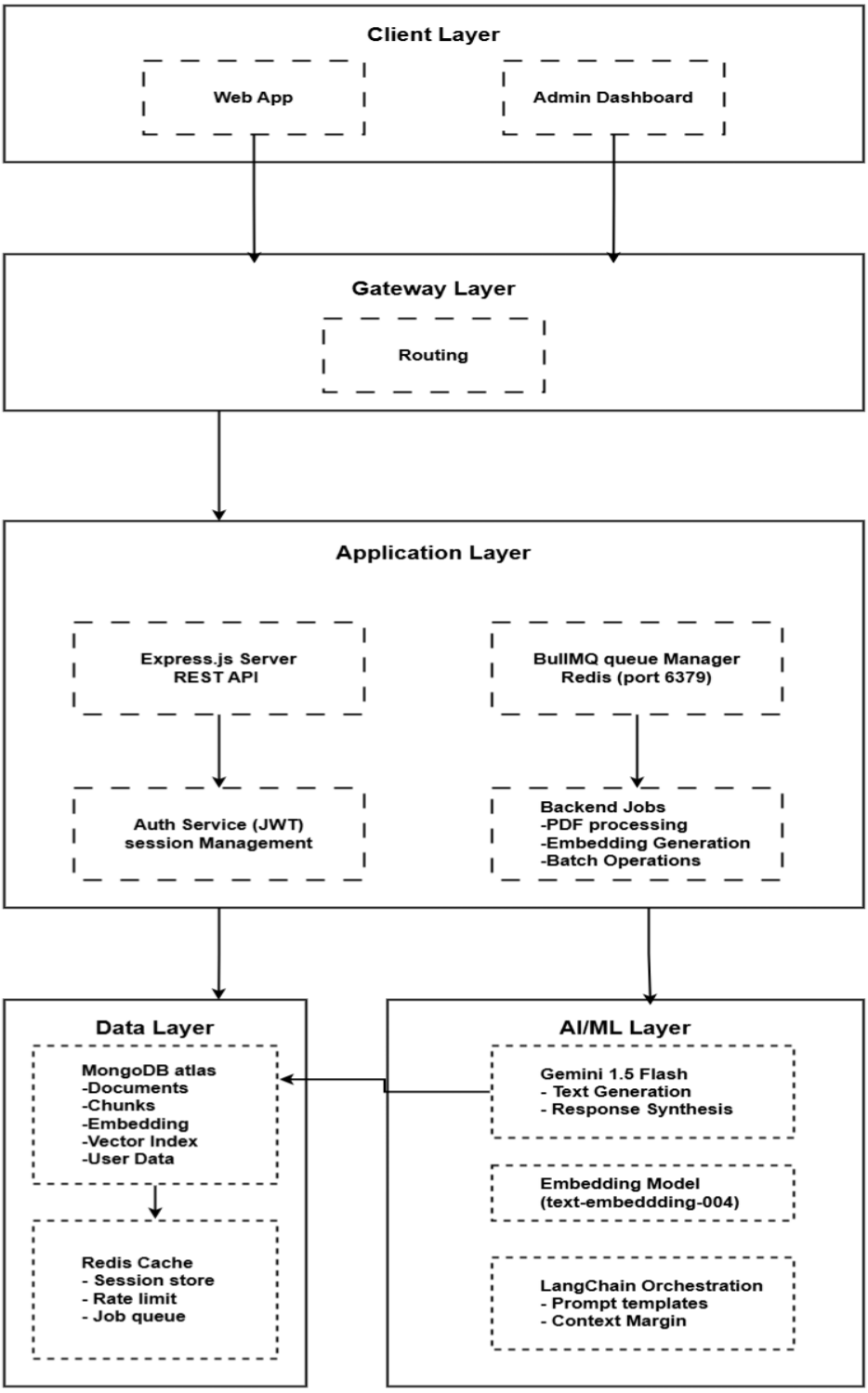
OpsMind AI: High-Level Design (HLD) Document

1. Document Control

Version	Date	Author	Description
1.0	February 16, 2026	Vaibhav Sutar	Initial HLD for OpsMind AI

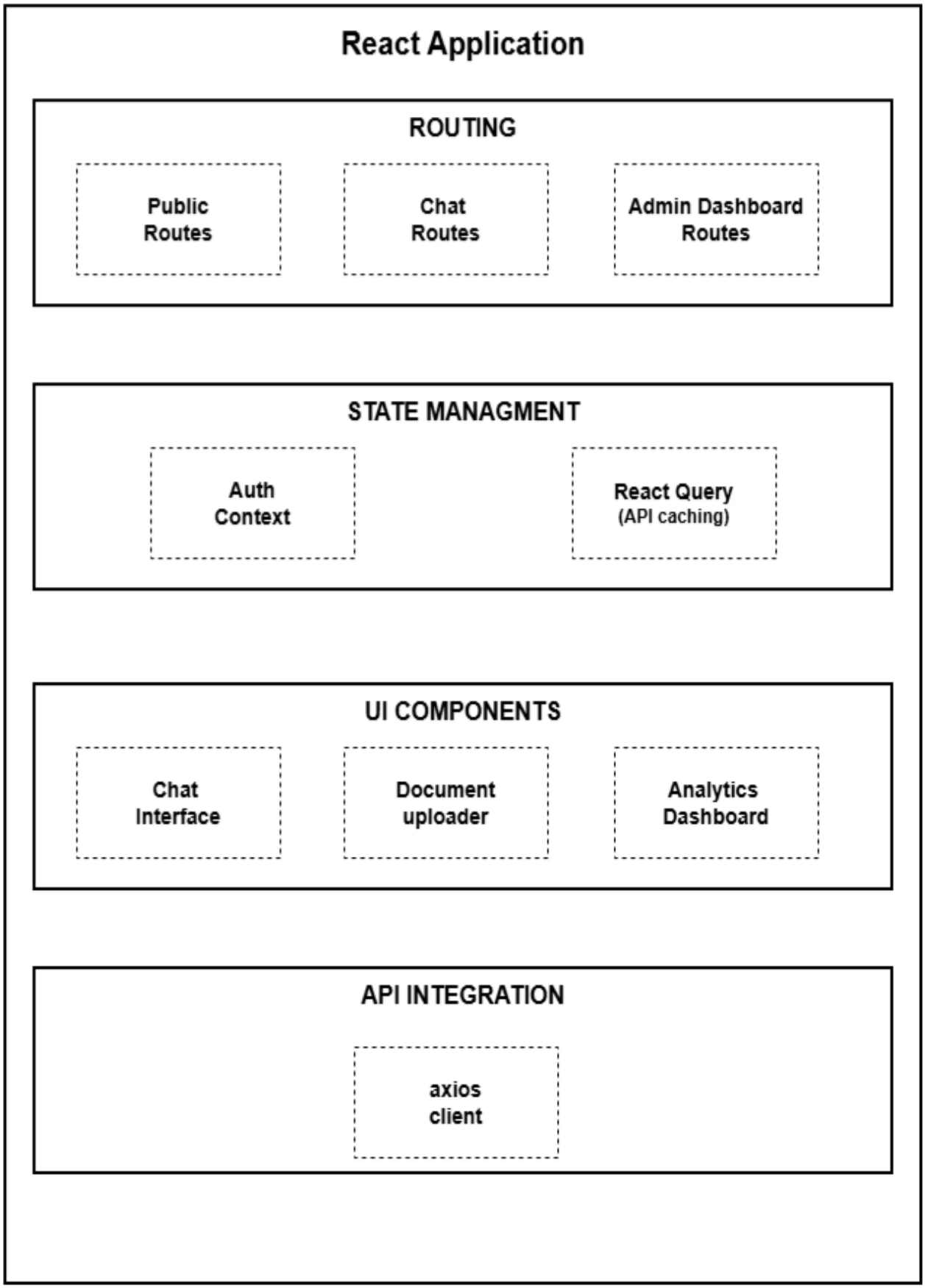
2. System Architecture Overview

a. High-Level Diagram

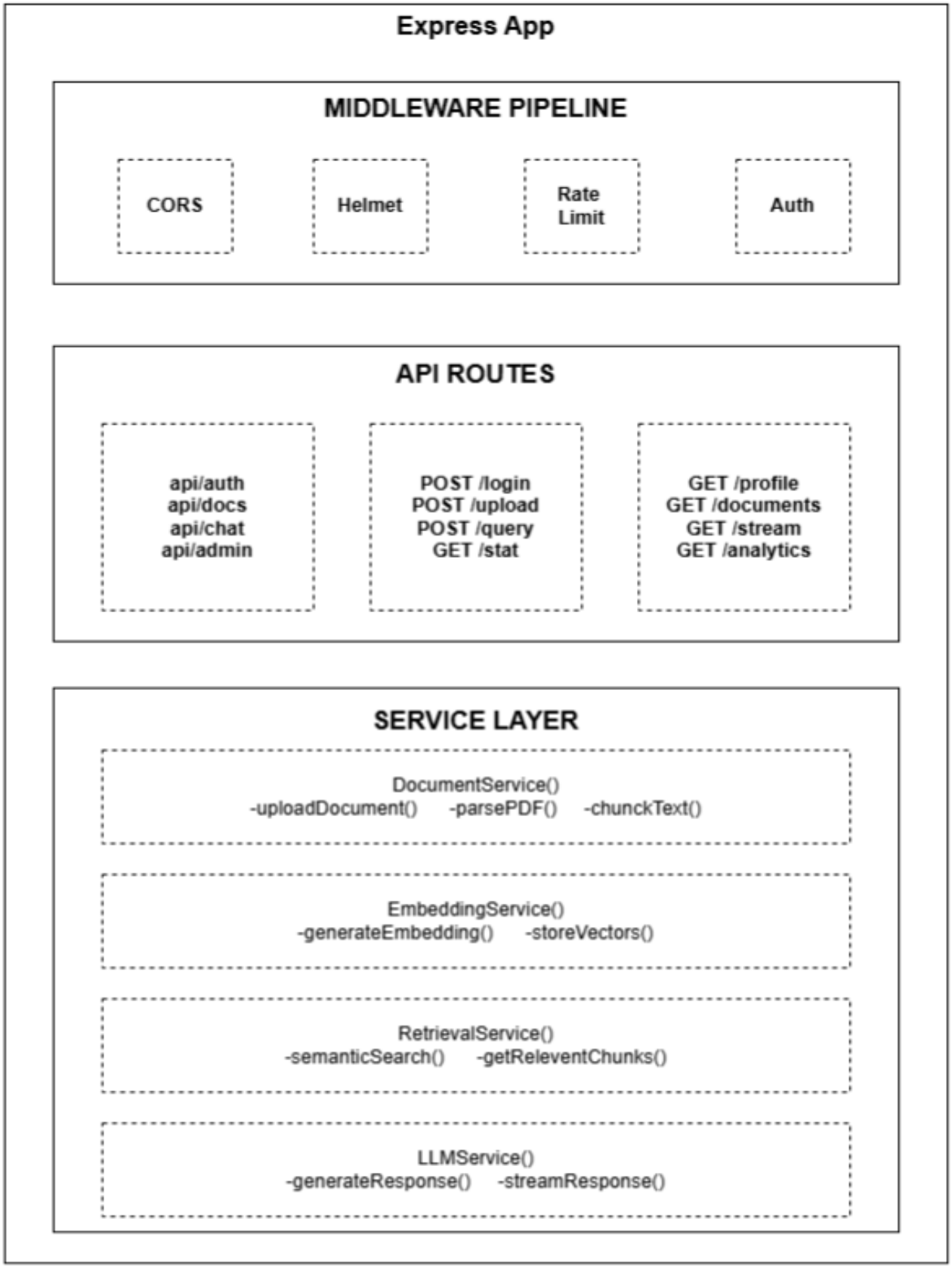


3. Component Architecture

3.1 Frontend Architecture



3.2 Backend Component



4. Data Architecture

4.1 **MongoDB Schema Design**

```
// Database: opsmind

// Collections:
// 1. documents - Original uploaded files
{
    _id: ObjectId,
    filename: String,
    originalName: String,
    mimeType: String,
    size: Number,
    uploadDate: Date,
    uploadedBy: ObjectId (ref: users),
    totalPages: Number,
    status: String // 'processing', 'processed', 'failed'
}

// 2. chunks - Text chunks with embeddings
{
    _id: ObjectId,
    documentId: ObjectId (ref: documents),
    text: String,
    textHash: String, // For deduplication
    chunkIndex: Number,
    pageNumber: Number,
    startPosition: Number,
    endPosition: Number,
    embedding: Array<Number>, // 768-dim vector
    metadata: {
        filename: String,
        section: String,
        heading: String
    },
    createdAt: Date
}

// 3. conversations - Chat history
{
    _id: ObjectId,
    userId: ObjectId (ref: users),
```

```
        query: String,
        queryEmbedding: Array<Number>,
        response: String,
        citations: [
            {
            documentId: ObjectId,
            filename: String,
            pageNumber: Number,
            chunkId: ObjectId,
            relevanceScore: Number
            }
        ],
        feedback: {
            helpful: Boolean,
            comment: String
        },
        timestamp: Date
}

// 4. users - User accounts
{
        _id: ObjectId,
        email: String,
        passwordHash: String,
        name: String,
        role: String, // 'admin', 'user'
        organization: String,
        createdAt: Date,
        lastLogin: Date
}

// 5. analytics - Usage statistics
{
        _id: ObjectId,
        date: Date,
        totalQueries: Number,
        uniqueUsers: Number,
        topDocuments: [
            {
            documentId: ObjectId,
```

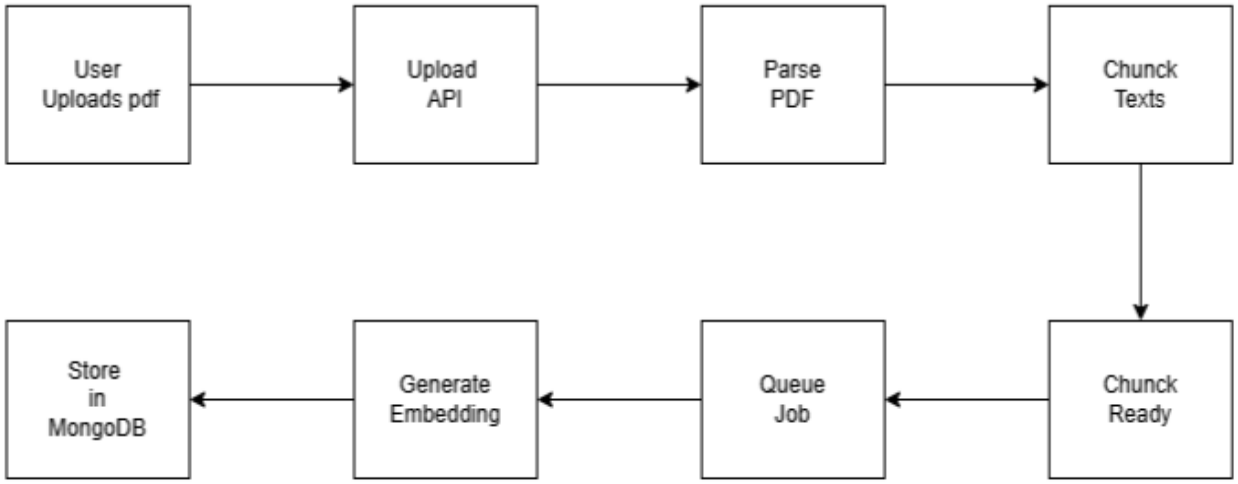
```
            queryCount: Number
            }
        ],
        unansweredQueries: [String], // Queries that got "I don't know"
        averageResponseTime: Number
}
```

4.2 **Vector Search Index Configuration**

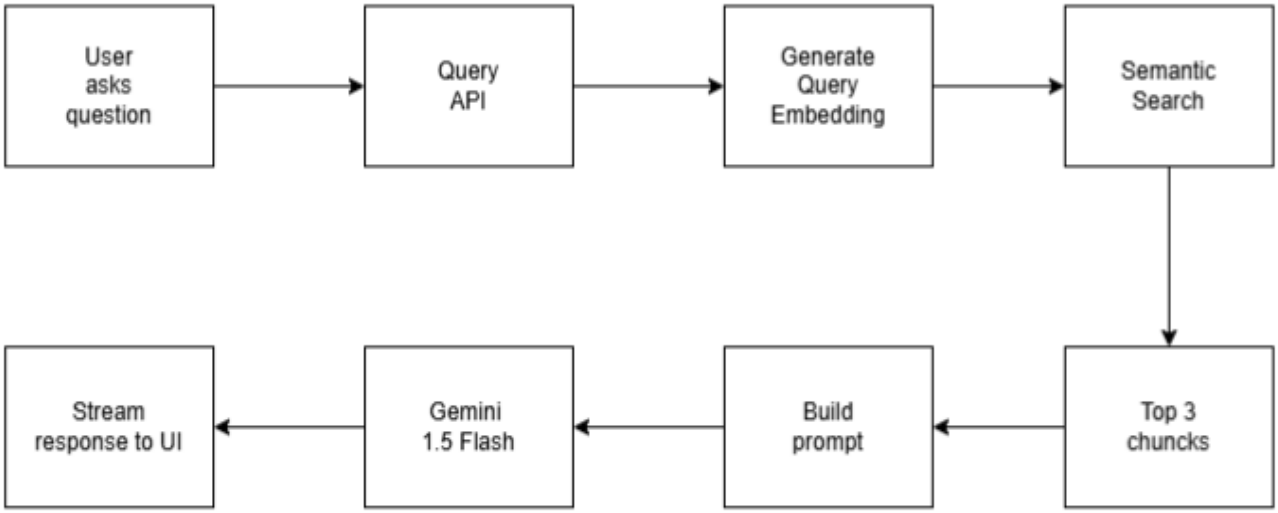
```
{
  "mappings": {
    "dynamic": true,
    "fields": {
      "embedding": {
        "dimensions": 768,
        "similarity": "cosine",
        "type": "knnVector"
      },
      "text": {
        "type": "string"
      },
      "documentId": {
        "type": "objectId"
      },
      "pageNumber": {
        "type": "number"
      }
    }
  }
}
```

5. Data Flow Diagrams

5.1 Document Ingestion Flow



5.2 Query Processing Flow



6. API Design

6.1 API Endpoints

Method	Endpoint	Description	Request Body	Response
POST	/api/auth/login	User login	{email, password}	{token, user}
POST	/api/auth/register	User registration	{email, password, name}	{token, user}
POST	/api/documents/upload	Upload PDF	multipart/form-data	{documentId, status}
GET	/api/documents	List Documents		[{document}]
DELETE	/api/documents/:id	Delete Documents		{success}
POST	/api/chat/query	Ask question	{query}	{response, citations}
GET	/api/chat/stream	Streaming query	?query=...	text/event-stream
GET	/api/admin/stats	Get analytics		{queries, docs, users}
POST	/api/chat/feedback	Submit Feedback	{conversationId, helpful}	{success}

6.2 Streaming API Specification

GET /api/chat/stream?query={encodedQuery}

Headers:

Authorization: Bearer {token}

Accept: text/event-stream

Response Stream Format:

```
event: message
data: {"text": "The response chunk..."}

event: citation
data: {"citations": [{"filename": "policy.pdf", "page": 5}]}

event: done
data: {}
```

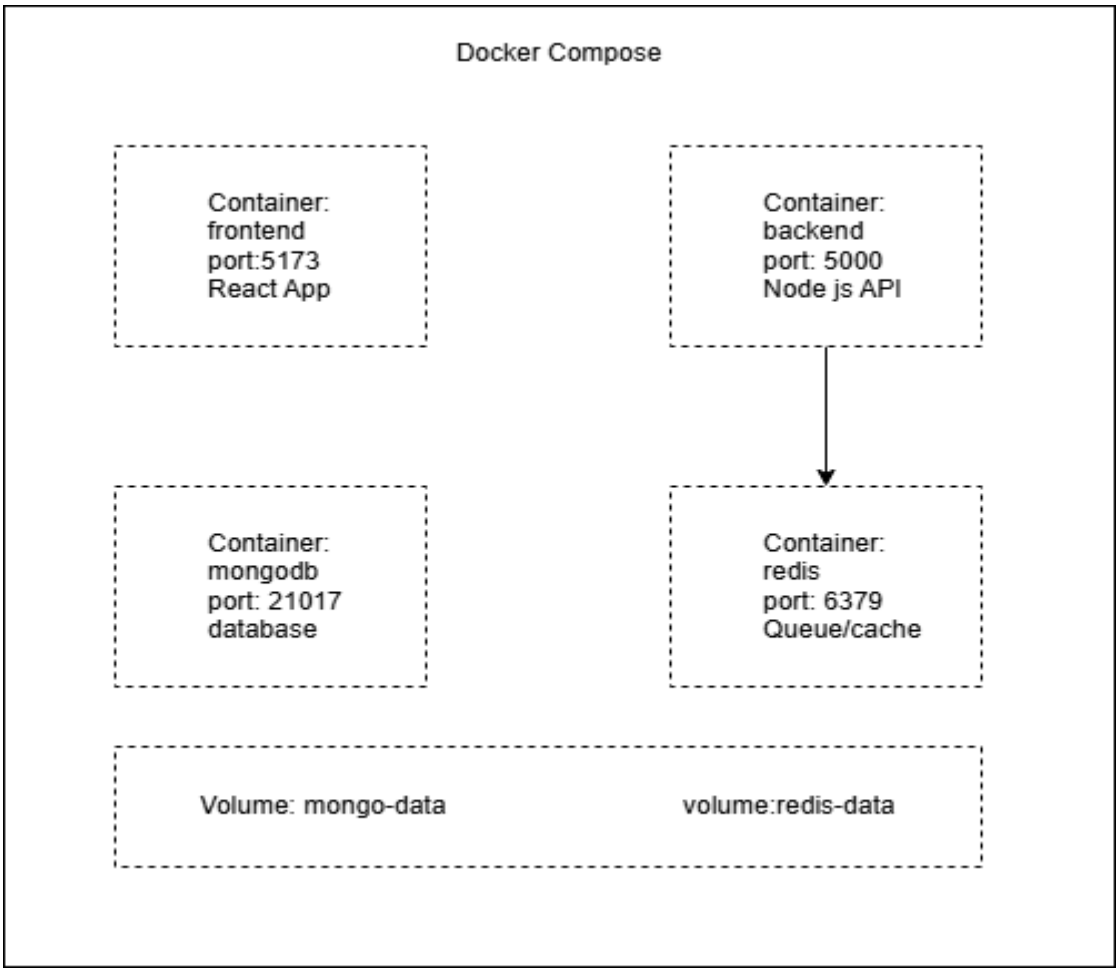
7. **Queue Architecture (BullMQ)**

```
// Queue Configuration
const PDF_QUEUE = {
  name: 'pdf-processing',
  concurrency: 3,
  redis: {
    host: 'redis-server',
    port: 6379
  }
};

const EMBEDDING_QUEUE = {
  name: 'embedding-generation',
  concurrency: 5,
  redis: {
    host: 'redis-server',
    port: 6379
  }
};

// Job Structure
// PDF Processing Job
{
  jobId: uuid(),
  type: 'process-pdf',
  data: {
    documentId: ObjectId,
    filePath: String,
    originalName: String
  },
  attempts: 3,
```

```
  backoff: {
    type: 'exponential',
    delay: 1000
  }
}
```

8. Deployment Architecture

8.1 Docker Container Structure



8.2 Environment Configuration

```
# Backend .env

NODE_ENV=production

PORT=5000


# Database

MONGODB_URI=mongodb://mongodb:27017/opsmind     ## atlas URI

MONGODB_DB_NAME=opsmind


# Redis

REDIS_HOST=redis

REDIS_PORT=6379
```

```
# AI Services
GEMINI_API_KEY=api_key
EMBEDDING_MODEL=text-embedding-004

# Security
JWT_SECRET= secrete_key
CORS_ORIGIN= frontend_domain

# Rate Limiting
RATE_LIMIT_WINDOW=15
RATE_LIMIT_MAX=100
```

9. Development Guidelines

9.1 **Git Branching Strategy**

```
main
└── develop
    ├── feature/pdf-ingestion
    ├── feature/vector-search
    ├── feature/chat-ui
    └── hotfix/critical-bug
```

9.10 **Code Organization**

```
opsmind-ai/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── hooks/
│   │   ├── utils/
│   │   └── App.js
│   ├── public/
│   └── package.json
│
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── services/
│   │   ├── middleware/
│   │   ├── utils/
│   │   ├── config/
│   │   └── app.js
│   ├── tests/
│   └── package.json
│
│
```

```
│
├── docker/
│       ├── frontend.Dockerfile
│       ├── backend.Dockerfile
│       └── docker-compose.yml
├── env.example
└── README.md
```